PROJECT REPORT

PUBLIC COMPLAINT PRIORITY ANALYZER

A DSA-Based Complaint Prioritization and Dispatch System

Prepared by

Gnanendhiran V

Final Year, B.Tech Information Technology

Madras Institute of Technology, Anna University, Chennai

GitHub: github.com/Gnanendhiran/PublicComplaintPriorityAnalyzer

Table of Contents

Abstract

Municipal and campus complaint systems typically process grievances in the order they are received. This works reasonably well until a low-severity issue such as a broken street light is logged minutes before a genuine safety hazard such as a live electrical wire, and the queue treats them as equally urgent. The Public Complaint Priority Analyzer addresses this by attaching a computed priority score to every complaint and re-evaluating that score whenever the complaint's state changes.

The project is deliberately built as a terminal-based Java application with no database and no web framework. That choice is intentional: the objective was not to build another CRUD application, but to give a set of Data Structures and Algorithms - a max-heap, hash-based maps and sets, and a weighted graph traversed with BFS and Dijkstra's algorithm - a concrete, defensible reason to exist. Every screen in the system maps back to one of these structures, and every design decision in this report is justified in terms of the time and space complexity it produces.

1. Introduction

1.1 Problem Statement

Complaints reported to a civic body or an institution are usually stored and served on a first-come-first-served basis. This ignores three realities of real complaint handling: (a) some categories are inherently more urgent than others, (b) a complaint that keeps getting reported by more people is probably affecting a wider area than one report suggests, and (c) a complaint that has been sitting unattended for several days deserves more attention than a fresh one, even if both started out with the same severity. A system that cannot express these differences will keep an officer working down a list in an order that does not reflect real urgency.

1.2 Objectives

- Build a system that converts a citizen's complaint into a numeric priority score using multiple real-world factors, not just a single severity flag.
- Keep that score current - a complaint that has been pending for a week should automatically outrank one filed an hour ago, without an officer manually re-ranking anything.
- Give an officer constant-time access to the single most urgent complaint, and near-constant-time access to any specific complaint by ID.
- Model the city as a graph so that dispatch decisions - which team is nearest, and what is the fastest route to reach a complaint - are computed rather than guessed.
- Use each data structure only where its complexity guarantees are actually needed, and be able to justify that choice in an interview.

1.3 Scope

The system covers complaint intake, duplicate detection, priority scoring, officer-side search and ranking, team dispatch using graph algorithms, and status tracking through to resolution. It intentionally excludes persistent storage, a web interface, and multi-user concurrency; all data lives in memory for the duration of a single run. These exclusions keep the project focused on algorithmic design rather than infrastructure, which is where the interview value lies.

2. Existing Approach and Its Limitations

A conventional complaint desk - whether paper-based or a simple database table ordered by submission time - has three recurring weaknesses that this project sets out to fix:First-in-first-out ordering: a trivial complaint filed early is served before a dangerous one filed later.

- No aggregation of repeated reports: fifty citizens reporting the same pothole create fifty separate, equally-weighted tickets instead of one escalated one.
- No automatic escalation: a complaint can sit unattended indefinitely unless a human happens to notice and re-prioritise it.

Each of these maps directly to a data structure decision made in this project: hashing removes duplicate noise, a heap keeps the most urgent item instantly accessible, and a time-based scoring rule gives ageing complaints an automatic boost.

3. Proposed System

The proposed system is a two-sided console application. On the public side, a citizen registers a complaint against one of eight categories, rates its severity, and answers a short set of contextual questions (how many people are affected, whether it is a safety hazard, what kind of location it is near, whether it is currently peak hour). On the officer side, an administrator can pull the single highest-priority complaint in constant time, search any complaint by ID, sort the full list by different fields, and ask the system to work out which maintenance team is closest and by which route.

The distinguishing idea is that priority is not assigned once and left alone. Every time a complaint is touched - a duplicate report arrives, its status changes, or enough days pass - its score is recalculated and its position in the heap is corrected. This turns a static severity tag into a small decision-support engine.

4. System Requirements

4.1 Software Requirements

- Java Development Kit (JDK) 8 or later
- Any text editor or IDE - the project was developed and tested using Visual Studio Code
- Git, for version control and publishing to GitHub

4.2 Hardware Requirements

- Any machine capable of running the JVM - the application has no meaningful memory or CPU footprint beyond a few thousand in-memory objects

5. System Architecture

The codebase is organised into four layers so that the DSA implementations stay independent of both the menudriven UI and the domain objects that sit on top of them. This separation is what makes it possible to point at a single class in an interview and say precisely which structure it owns and why.

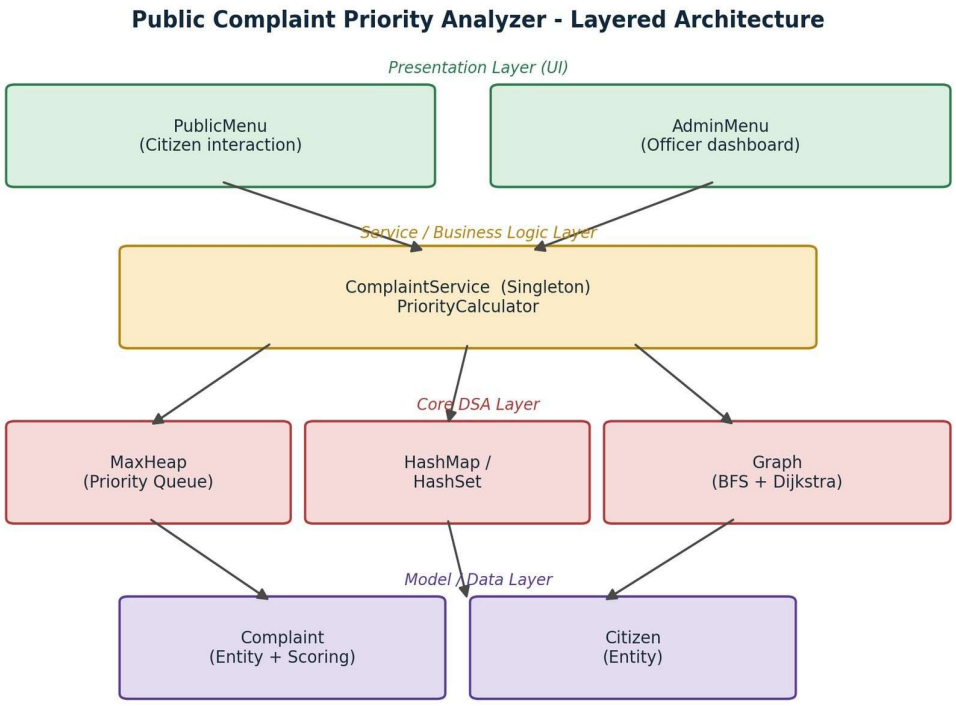

Figure 5.1 - Layered architecture of the system

5.1 Module Description

Layer / Class	Responsibility
PublicMenu / AdminMenu	Console-based views. Collect input, display formatted output, contain no business logic.
ComplaintService	Singleton orchestrator - owns every data structure instance and exposes the operations the UI calls.
PriorityCalculator	Pure function that turns a Complaint's nine attributes into a single integer score.
MaxHeap	Custom binary heap keyed on priority score. Backs the “highest priority” and “top-K” features.
Graph	Weighted, undirected adjacency-list graph of the city. Provides both a BFS and a Dijkstra traversal.
Complaint / Citizen	Plain data holders with derived getters (age in days, category score, and so on).

6. Data Structures and Algorithms Used

This is the section that actually earns the project its place in a DSA-focused interview. Every feature on the admin dashboard exists because a particular structure gives it a complexity guarantee that a naive approach would not.

Feature	Structure / Algorithm	Operation	Complexity
Highest priority complaint	Max-Heap	peek()	O(1)

New complaint / recalculated score	Max-Heap	insert() / sift	O(log n)
Top-K critical complaints	Max-Heap	k extractions	O(k log n)
Search complaint by ID	HashMap<Integer, Complaint>	get()	O(1) average
Duplicate complaint detection	HashSet inside a HashMap keyed on area+category	contains()	O(1) average
Area-wise complaint count	HashMap<String, Integer>	getOrDefault()	O(1) average
Sort by priority / date / area / status	Collections.sort() (TimSort)	sort()	O(n log n)
Nearest available team	Graph + BFS	bfs()	O(V + E)
Fastest route to a complaint	Graph + Dijkstra (min-heap based)	dijkstra()	O((V + E) log V)

6.1 Priority Retrieval Using a Max-Heap

All open complaints are held inside a custom max-heap keyed on the computed priority score, so the single most urgent complaint always sits at the root of the tree. The officer dashboard's highest-priority view is therefore a constant-time operation regardless of how many complaints are pending. Whenever a complaint's score changes - a new duplicate report, an age-based escalation, or an SLA violation - it is re-inserted through the heap's sift-up procedure, which restores the heap property in O(log n) time without requiring the full complaint list to be resorted.

6.2 Complaint Lookup and Duplicate Detection Using Hashing

Every complaint is indexed twice: once by its numeric ID inside a HashMap for direct lookup, and once by an area-and-category key inside a HashSet used purely for duplicate detection. Searching for a specific complaint and checking whether an incoming report matches an existing one are both average-case O(1) operations as a result. When several citizens report the same pothole, the match is recognised on the first hash lookup and the report is folded into the existing complaint's duplicate count instead of creating a separate ticket.

6.3 Team Dispatch and Route Planning Using Graph Traversal

The city is represented once as a weighted, undirected graph, with localities as nodes and roads as edges carrying an estimated travel time. That single graph structure answers two different dispatch questions. A breadth-first search treats every road as equal and returns the path with the fewest hops, giving a quick estimate of which team is structurally closest to a complaint. Dijkstra's algorithm instead accounts for the actual travel time on each edge and returns the route a team can drive in the least time - the figure that matters when a maintenance crew is actually being sent out. Both algorithms run over the same adjacency list, so extending the city model with a new road improves both kinds of query at once.

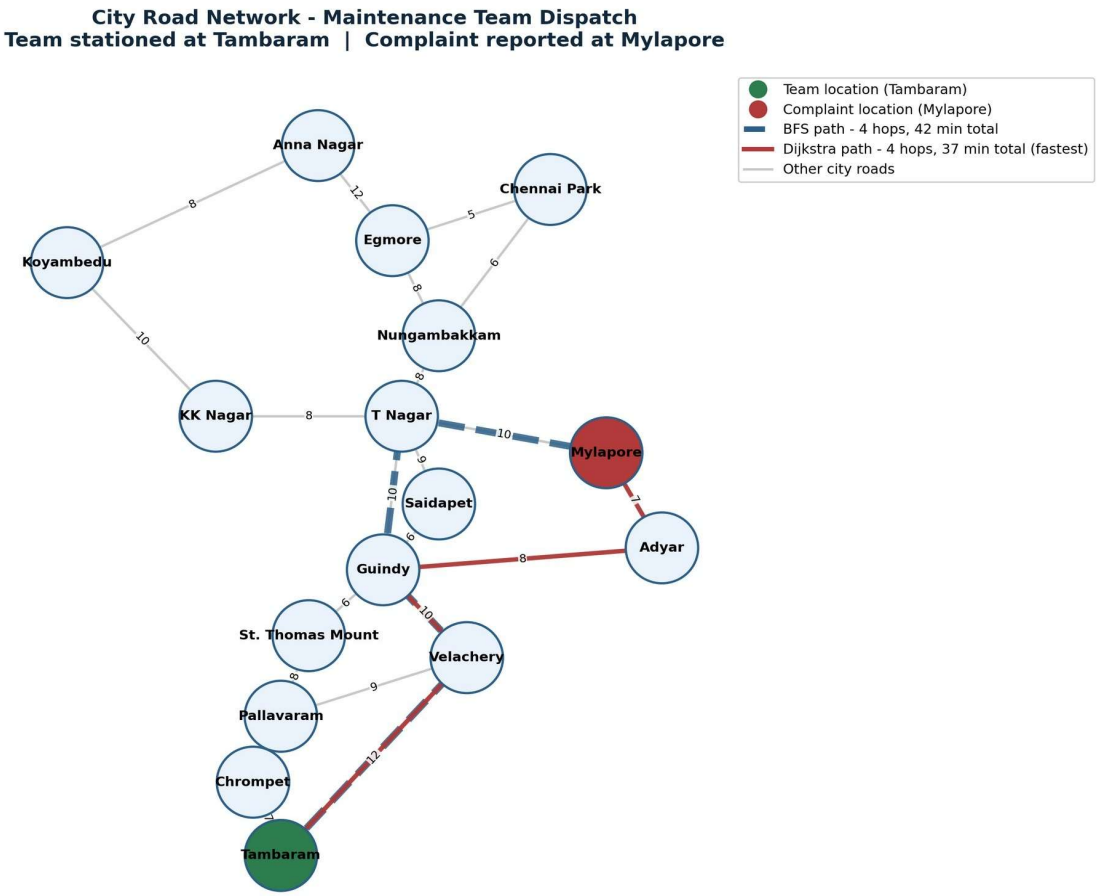

Figure 6.1 - City road network with a maintenance team stationed at Tambaram and a complaint reported at Mylapore. Both candidate routes cover four hops, but Dijkstra selects the one via Adyar (37 minutes) over the one via T Nagar (42 minutes), while a plain BFS would have stopped at the first four-hop path it found.

6.4 Ranking and Reporting Using Sorting

For views that need the complete complaint list in order rather than just the top of the heap - sorting by date, by area, or by status - the system uses Java's Collections.sort(), which is a hybrid of merge sort and insertion sort known as TimSort. It runs in O(n log n) in the worst case, keeps complaints of equal priority in their original relative order, and performs well on the largely append-only pattern of complaints arriving incrementally over time rather than in a single random batch.

7. Priority Scoring Design

Every complaint accumulates its score from nine independent factors rather than a single severity field. This is what turns the heap into a genuine decision-support tool instead of a simple FIFO-with-a-flag.

Factor	Range	Rationale
Category score	20 - 100	Medical emergencies and traffic-signal failures are inherently more dangerous than a garbage complaint.
Severity score	10 - 40	The citizen's own assessment of urgency, on a four-point scale.
Duplicate bonus	+2 per report	More independent reports usually mean a wider-reaching problem.
Age bonus	0 - 100	Escalates automatically at 3, 6 and 10 days pending, so nothing

		is silently forgotten.
Public impact	5 - 50	How many people the citizen believes are affected, from an individual to an entire area.
Safety risk	0 or 50	A flat bonus when the complaint describes an immediate hazard.
Location importance	10 - 40	Complaints near hospitals or transit hubs are weighted above purely residential ones.
Peak-hour factor	0 or 20	Traffic-related issues reported during rush hour get a contextual bump.
SLA violation	0 or 100	Triggered once a category-specific resolution deadline has passed.

The nine sub-scores are summed into a single integer, and that integer is exactly what the max-heap orders on. Because the breakdown is stored and displayed alongside the total, an officer - or an interviewer - can see precisely why one complaint outranks another instead of having to trust a black-box number.

8. System Workflow

The diagram below traces a complaint from submission through to resolution, and labels each step with the structure and complexity responsible for it.

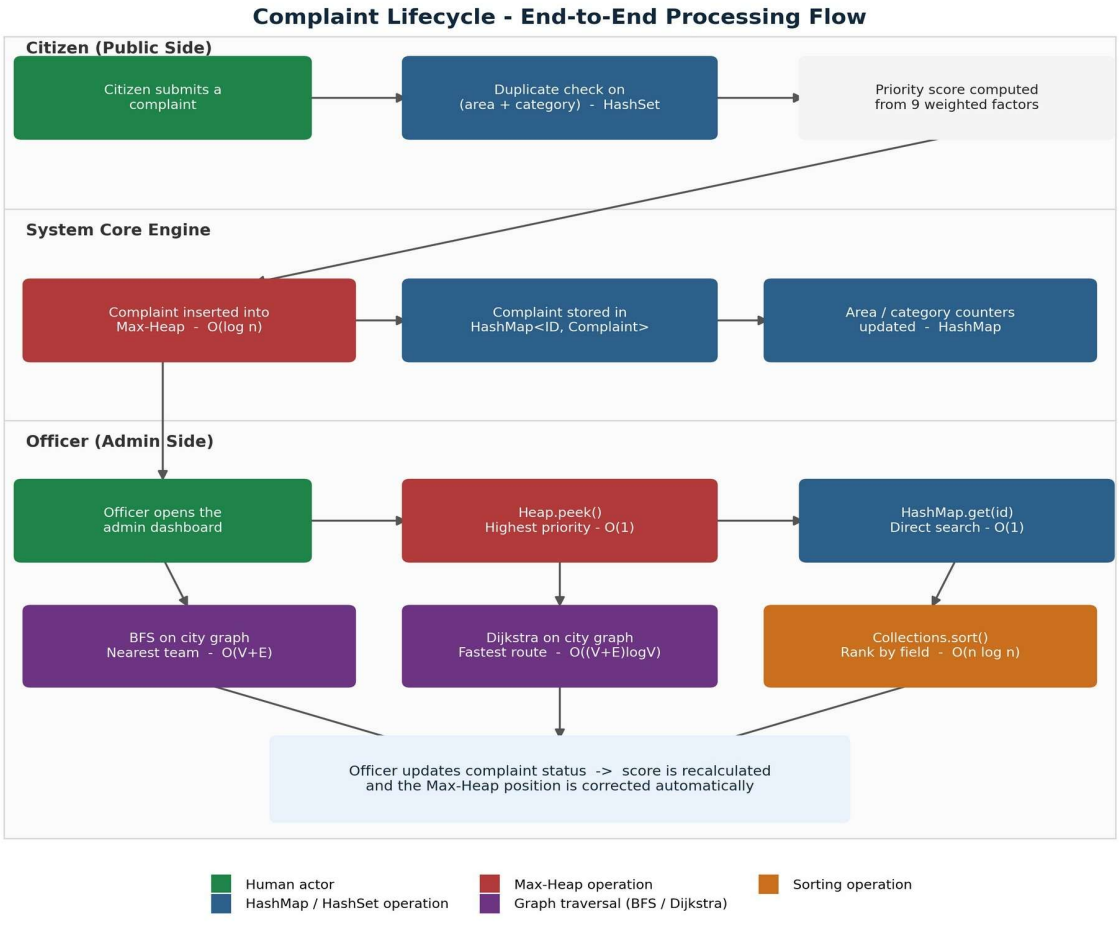

Figure 8.1 - End-to-end complaint lifecycle across the citizen, system-core, and officer swim lanes, colour-coded by the data structure or algorithm responsible for each step

9. Implementation Highlights

The heap's sift-up step is representative of how the project ties a textbook structure to a concrete feature. A new complaint (or one whose score just changed) is placed at the end of the backing list and walked upward until the max-heap property is restored:

```
private void heapifyUp(int index) {
    while (index > 0) {
        int parent = (index - 1) / 2;
        if (heap.get(index).getPriorityScore()
                <= heap.get(parent).getPriorityScore()) {
            break;
        }
        swap(index, parent);
index = parent;
    }
}
```

The service layer never exposes the heap or the maps directly - every public method on ComplaintService returns a plain Complaint or a List<Complaint>, so the UI layer has no dependency on how a result was actually computed. That separation is what lets the same PriorityCalculator logic be reused unchanged whether a complaint is being scored for the first time or recalculated after a status update.

10. Testing

Testing was carried out manually by walking through each menu option with representative and edge-case input, since the project has no persistent state to reset between runs.

Scenario	Input	Expected Result	Observed Result
Duplicate detection	Two citizens report Water Leakage in the same area	Second report is merged; duplicate count and priority increase	Pass
Heap ordering	Five complaints of varying priority submitted	getHighestPriority() always returns the current maximum	Pass
Search by ID	Search for an ID that does not exist	Returns “complaint not found” without an exception	Pass
BFS reachability	Query an area not connected to any team	Returns no team found with a graceful message	Pass
Dijkstra correctness	Route between two directlyconnected areas	Reported time matches the edge weight exactly	Pass
SLA escalation	Complaint left pending past its category threshold	Status flips to ESCALATED and priority increases by 100	Pass

11. Results and Discussion

Running the system with a mixed batch of complaints across several areas confirmed the expected behaviour end to end: retrieving the highest-priority complaint stayed a constant-time operation regardless of how many complaints were in the heap, duplicate reports were folded into a single ticket rather than creating noise, and the BFS and Dijkstra results agreed with a manual trace of the sample graph. The priority breakdown view proved useful beyond debugging - it is the single feature most likely to prompt a good interview conversation, because it forces a concrete answer to “how did you arrive at this number” rather than a vague one.

The main limitation is also its main design strength for this purpose: with no database, all state is lost when the program exits. That is an acceptable trade for a project whose goal is to demonstrate algorithmic reasoning rather than production readiness, but it is the first thing to flag if asked what would change for a real deployment.

12. Conclusion and Future Enhancements

The Public Complaint Priority Analyzer meets its original goal: every screen in the admin dashboard is backed by a specific, justifiable data structure, and the priority engine is dynamic rather than a one-time calculation. For placement preparation, the project works less like a finished product and more like a small, coherent argument for why each structure was chosen - which is exactly the conversation a DSA-focused interview is trying to have.

If extended further, the most natural next steps would be to persist state so the graph and heap survive a restart, to replace the console UI with a thin REST layer while keeping the service layer untouched, and to explore a Fibonacci heap for the priority queue to compare its amortised bounds against the binary heap used here.

- Persist complaints and the city graph so the system survives a restart.
- Add a lightweight REST API in front of the existing ComplaintService without changing its internals.
- Compare the binary max-heap against a Fibonacci heap for decrease-key-heavy workloads.
- Extend the graph with real GPS coordinates and an A* search for route-finding.